

Copilot & VS Code 10 Hours

Master AI-powered development in this comprehensive video course where every line of code is created live using cutting-edge AI tools. Elevate your programming efficiency and unlock new coding possibilities.

Course Structure & Navigation

10 Modules

5 Videos per module · Hand-outs + Download-Repos.

Bite-sized Videos

Each video ≤ 12 Min → easily schedulable.

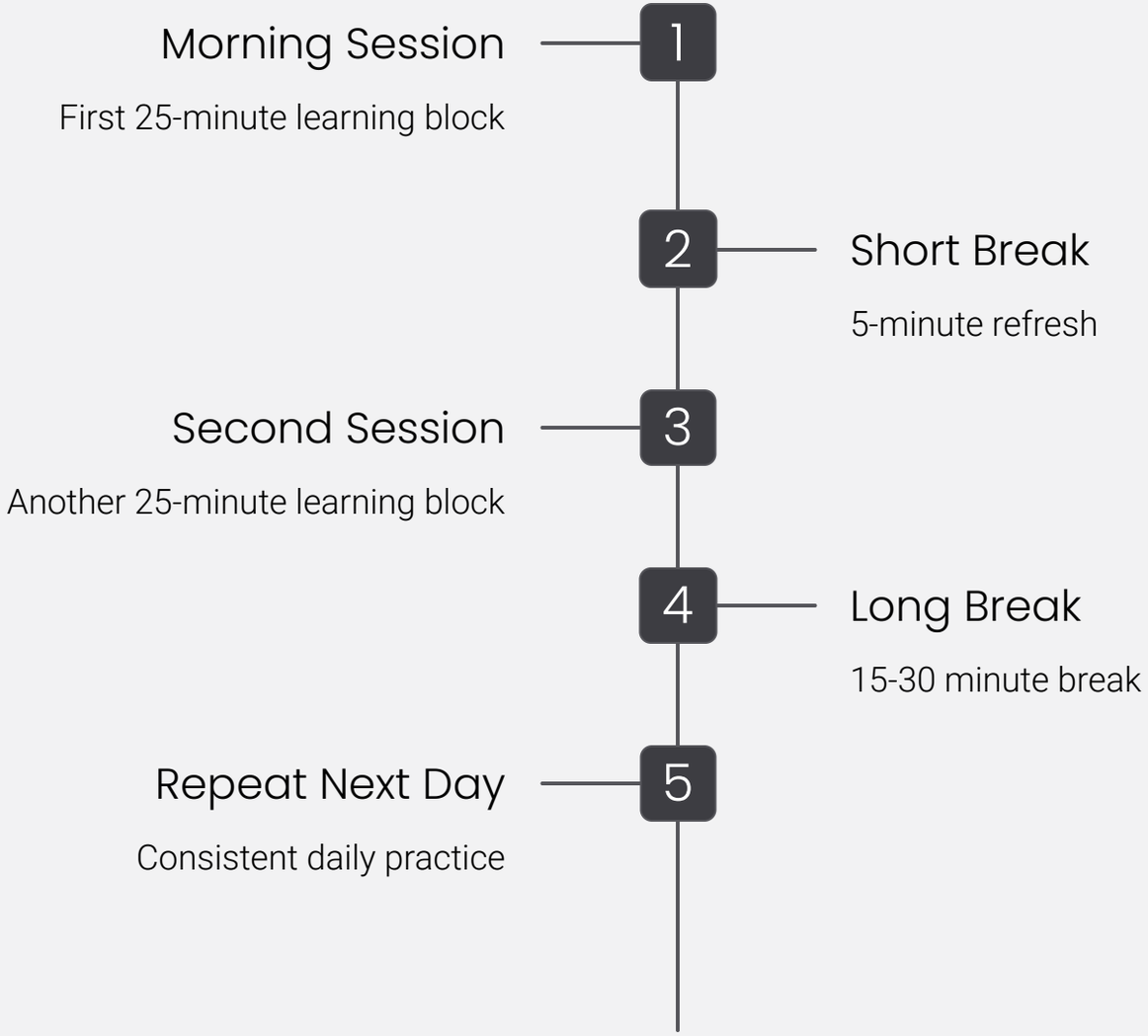
Self-Assessment

After each module: Self-Quiz (PDF + Google Form).

Practical Learning

Practice phases: Code along! – Use pause button, source code available.

Recommended Learning Schedule



The course is designed for optimal learning with 2 × 25 Min-blocks per day, following the Pomodoro technique for maximum retention and productivity.

Technical Requirements



VS Code ≥ 1.99

Latest version of Visual Studio Code



Git CLI

Command Line Interface for Git



GitHub Account

Free account for repositories and collaboration



Recommended Extensions

Copilot / Chat, GitLens, Live Server



Optional

Node 20 LTS + Python 3.12

Repo-Template & .devcontainer links provided in course materials.

Self-Paced Learning Tips

Optimize Your Setup

Two-screen setup: Video on one screen | VS Code on the other for optimal learning experience.

Use Chapter Markers

Chapter markers for quick review and revisiting specific concepts.

Check Solution Code

Source code diffs after each demo in branch /solutions for reference.

Get Help When Needed

FAQ-Doc & Discord-Forum for asynchronous questions and community support.



Git Snapshot Principle (Module 1)

Complete Snapshots

Commit = File system snapshot

Git doesn't store "deltas" but complete trees of your project at each commit point.

Foundation for Git Features

This snapshot approach is the basis for branching & revert functionality.

Enables powerful version control capabilities.

Visualization

Demonstrated in the video with git log --graph command.

See the commit history as a visual tree structure.

Git Workflow Basics

Initialize

Local repo initialization (git init)

Stage

Staging Area: add, status, diff

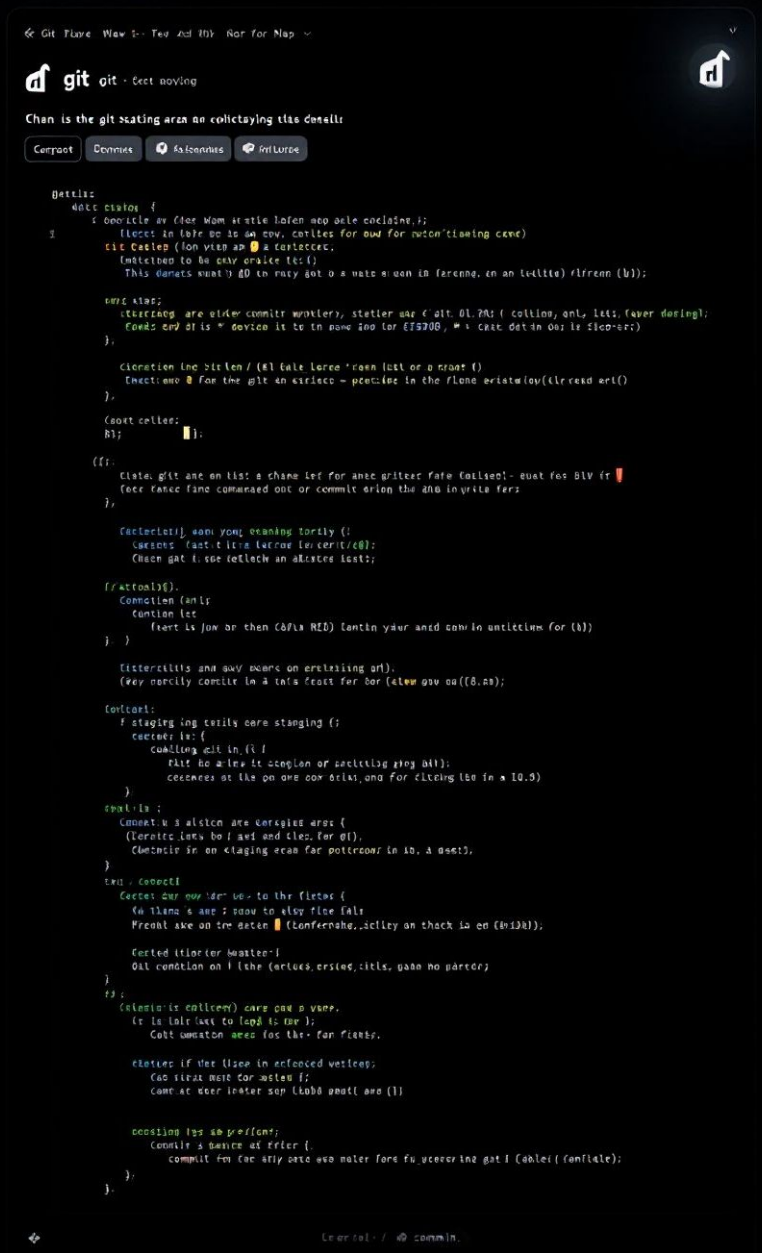
Commit

First commit with message

Push

Push to GitHub remote

Demo-Video: 4 Min | Pause-&-Code-along. Follow along with the instructor and practice these fundamental git commands.



Branching Strategies

Branch Types

Feature-Branch vs Main

Separate development work from stable code

Naming Convention

"feature/xyz" format

Clear identification of branch purpose

Integration Methods

Merge vs Rebase (animated graphic)

Different approaches to combining code

Checkpoint-Quiz at the end of the video to test your understanding of branching concepts.

Pull Requests in GitHub UI

Create Pull Request

PR creation process and assigning reviewers to your code changes.

Generate Description

Use Copilot to automatically generate PR descriptions based on your changes.

Review Process

Status checks & Squash-Merges for clean commit history.

Complete Merge

Finalize the process with proper merge strategy.

Screen-Capture-Demo shows the complete workflow from creating a PR to successful merge.

Git Cheatsheet & Resources

PDF "Pro Git Quick Reference"

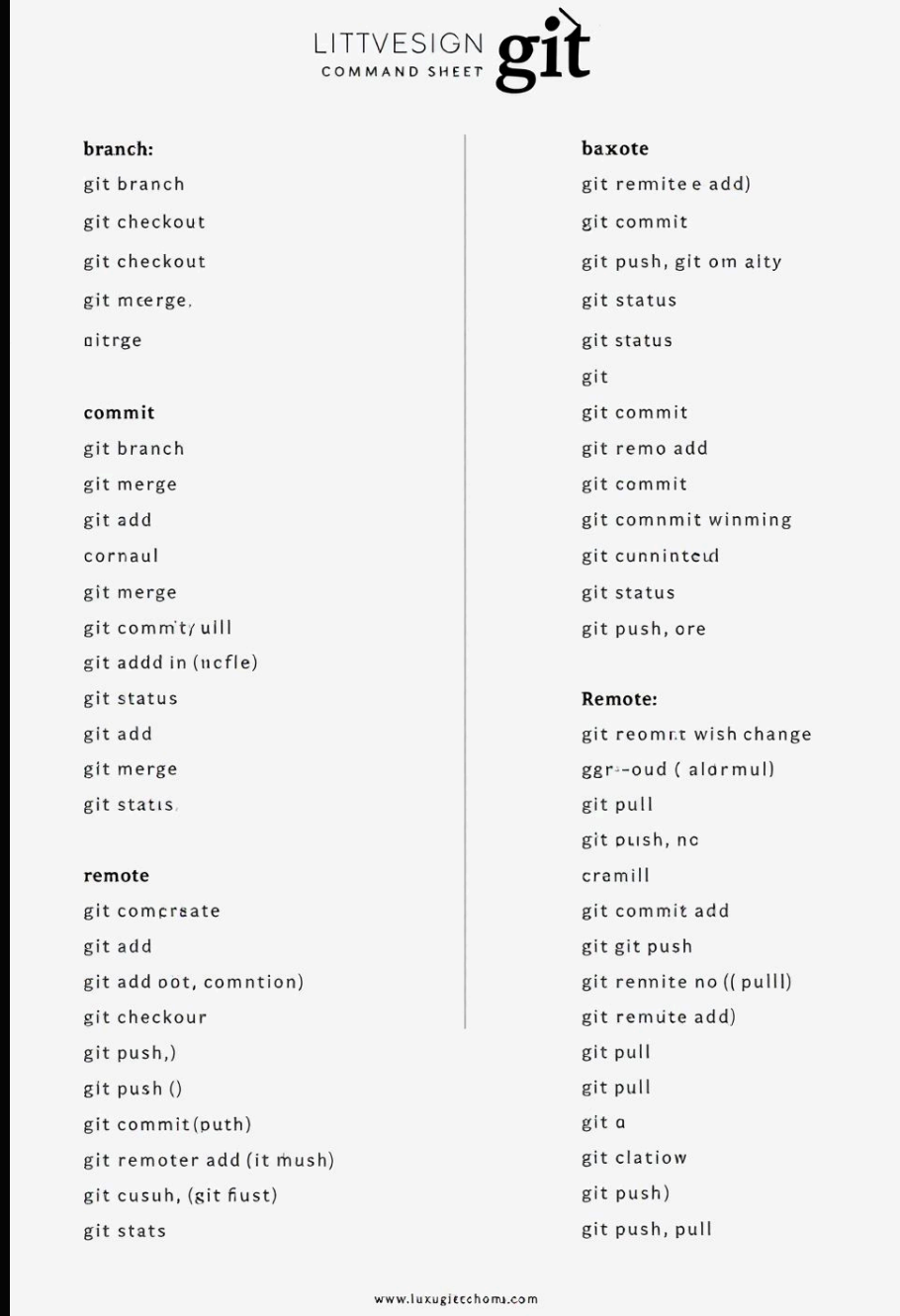
Downloadable cheatsheet with essential commands

Interactive Labs

GitHub Skills > "Intro to GitHub"

Advanced Automation

Recommendation: gh CLI for advanced automation



Module 1 Recap & Mini-Quiz

- Question 1: What does a Git commit represent?

  A complete snapshot of the file system, not just the changes (deltas).

- Question 2: What is the recommended naming convention for feature branches?

  "feature/xyz" format to clearly identify the branch purpose.

- Question 3: What are the two main methods to integrate changes from a branch?

  Merge and Rebase, each with different approaches to combining code.

- Question 4: What command shows a visual representation of the commit history?

  git log --graph

- Question 5: What is the staging area used for?

  To prepare and review changes before committing them to the repository.

Solution sheet included in the appendix. Next step: Start Module 2.

GitHub Actions Overview (Module 2)

CI/CD Benefits

Why Continuous Integration/Continuous Deployment?

Automated testing and deployment for reliable software delivery.

YAML Structure

YAML Basics: name, on, jobs

Key components of workflow configuration files.

Runner Options

Runner-Matrix vs Self-Hosted Runner

Different environments for executing your workflows.

First Pipeline in 8 Minutes



Select Template

Workflow-Template "Node CI"



Commit Configuration

Add workflow YAML file to repository



Action Runs

Automated workflow execution on push



Add Status Badge

Badge in README shows build status

Video-Demo + Pause-Marker. Follow along to create your first CI pipeline.

Secrets & Environments

GitHub Secrets UI

Secure storage for sensitive information

Repository and organization level secrets

Using Secrets in Workflows

Einsatz von ${{ secrets.API_KEY }}

Secure reference without exposing values

Enterprise Best Practices

Recommended approaches for managing secrets in organizations

Role-based access control and auditing

Codespaces & DevContainers



Cloud-VS Code in < 30 Sek

Instant development environment in your browser



.devcontainer.json Structure

Configuration file for development containers



Hot-Reload

Video demonstrates hot-reload functionality in browser



Collaboration

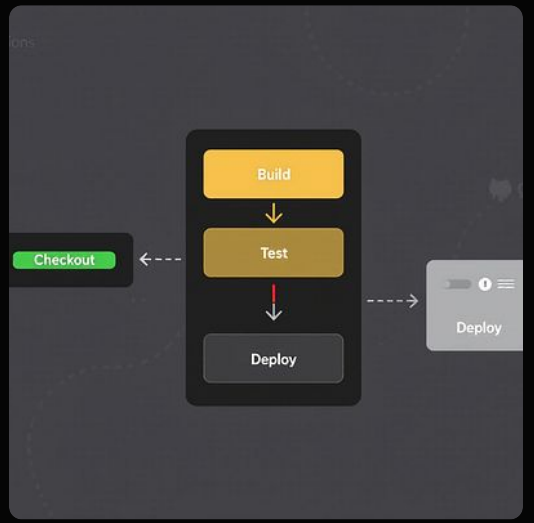
Share development environments easily

Module 2 Quiz & Resources









PDF: Actions Cheatsheet. Link-List to Starter Workflows. Let's continue with Copilot Basics.

Copilot Installation (Module 3)

Find Extension

Extension Marketplace → Install → Sign-in

Configure Settings

Telemetry Opt-in/out options for privacy preferences

Enterprise Setup

API-Key only needed for Enterprise-Proxy configuration

Verify Installation

Check status indicator to confirm Copilot is active

Inline-Completions in Action

Activation Methods

Tab-Trigger & Ghost Text

How to accept and navigate suggestions

Practical Example

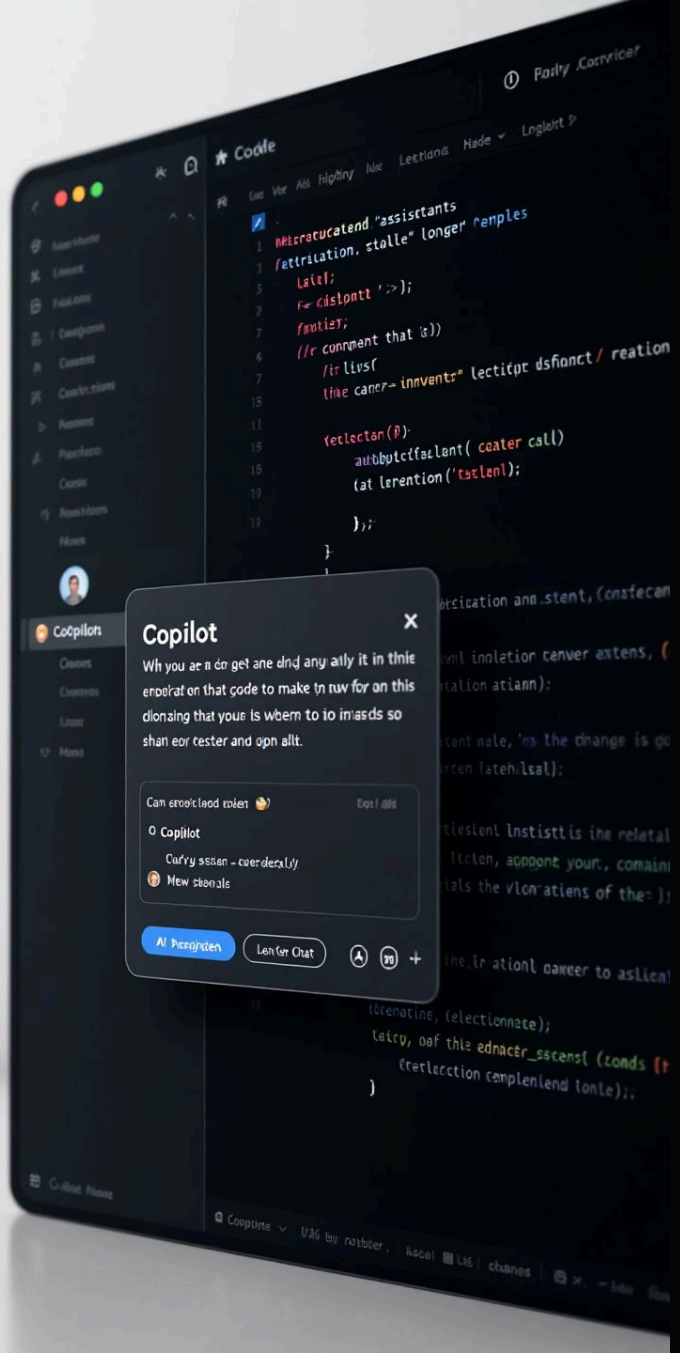
Demo-Snippet: JS-Array → Map/Reduce

See Copilot suggest complex array operations

Interactive Learning

Pause-&-Nutzt-die-Kommentare!

Use comments to guide Copilot's suggestions



Copilot Chat Fundamentals

Open Chat Panel

Chat-Panel öffnen (^⇧I / ⌘⇧I)

Use Slash Commands

Slash-Commands: /explain, /tests

Structure Your Prompts

Prompt-Design-Formel: **Role · Goal · Context**

Review & Refine

Iterate on responses for better results

Prompt-Recipe Demo

Define Goal

Ziel: "Generate Express CRUD Route"

Clear objective for the AI assistant

Set Constraints

Constraints: No ORM, Use Validations

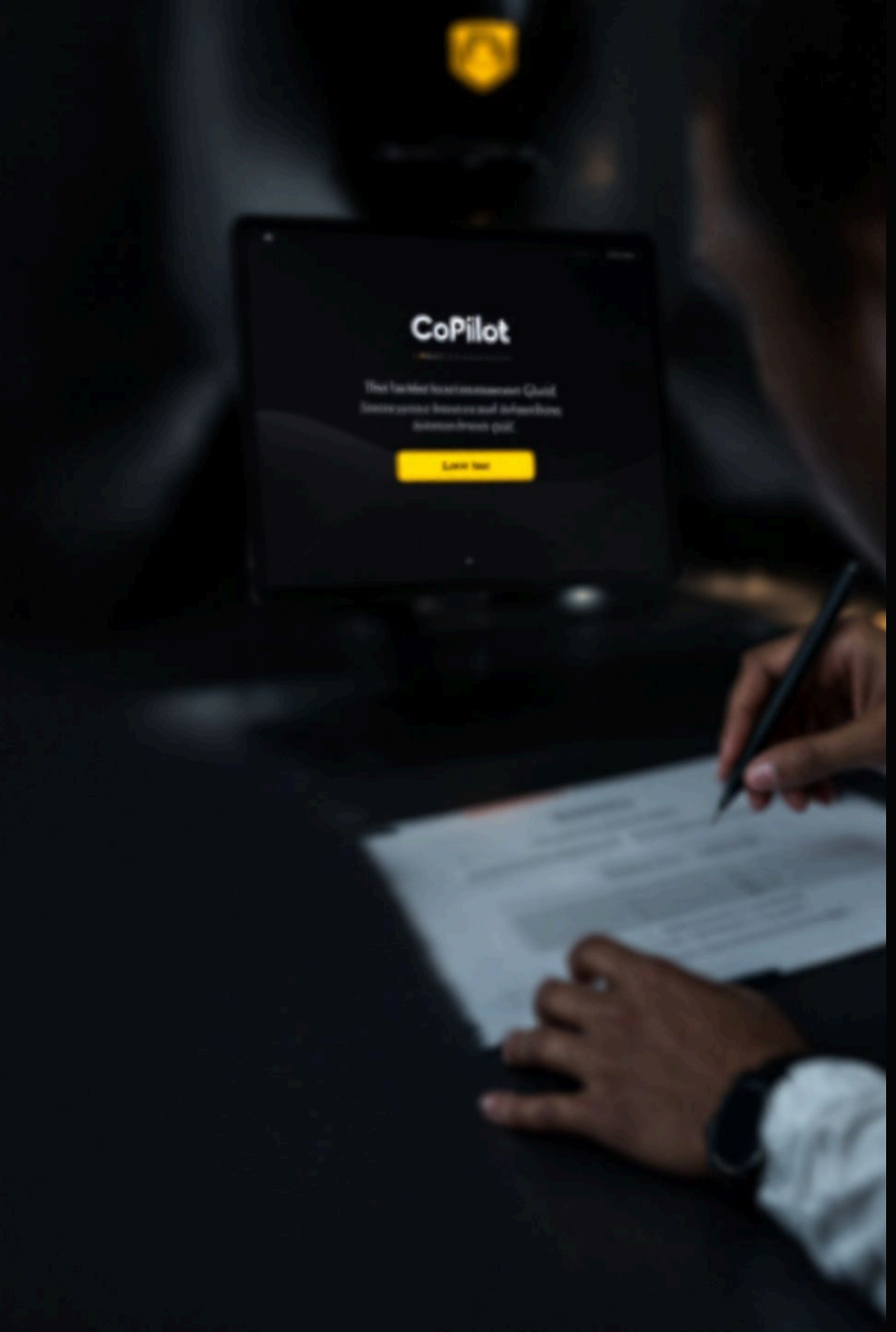
Specific requirements for the generated code

Review Output

Ergebnis + Diff-Anzeige im Video

See the generated code and changes

The video demonstrates how to craft effective prompts that produce exactly the code you need.



Module 3 Recap & Self-Quiz

6

Reflection Questions

Self-assessment questions to test your understanding

1

Cheatsheet

Download-Prompt-Cheatsheet.pdf for quick reference

3

Key Concepts

Core Copilot features covered in this module

Agent-Mode Introduction (Module 4)

/agent vs Chat

Differences between agent mode and regular chat

When to use each approach for different tasks

Plan-Preview & Approvals

Review generated plans before execution

Approve or modify proposed actions

Safety Considerations

Sicherheits-Disclaimer (Preview-Feature)

Understanding the limitations and risks

Multi-Step Task: OAuth Flow

Define Task

Specify OAuth flow implementation requirements

Generate Files

Agent erzeugt 4 Files with implementation

Run Tests

Automatically execute tests to verify functionality

Review Results

Examine generated code and test results

Bildschirm-Recording (8 Min). Empfehlung: Erst Dry-Run, dann Apply.



Copilot Workspace for PR Plans

Plan Creation

KI erstellt Plan im Browser → VS Code

Review Process

Review-Thread in Plan-Markdown

Implementation

Execute plan steps in VS Code

Verification

Confirm all plan items are completed

Demo-Video; Teilnehmer sollen Plan .md öffnen.

Creating Your Own Agent (Hello-World)

Manifest Structure

Minimal-Manifest erklären

Essential components of an agent definition

Tools Configuration

Tools-Sektion: pytest, npm test

Defining capabilities for your agent

Local Execution

Agent lokal über CLI anstoßen

Testing and running your custom agent



Module 4 Resources & Quiz

Code Samples

YAML-Snippet-Download

Assessment

5 Fragen zur Agent-Architektur

Documentation

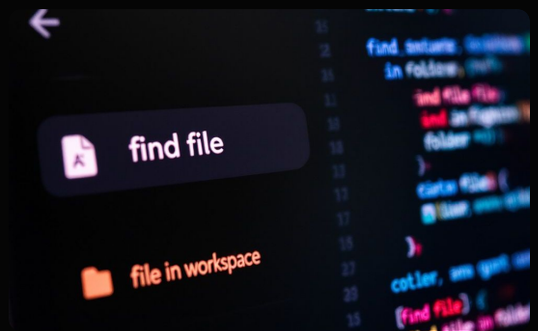
Docs-Link "Build a Copilot Agent"

Further Learning

Advanced agent development resources

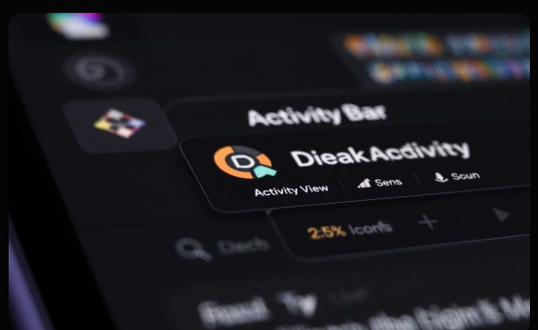


VS Code UI Tour (Module 5)



Command Palette

Access all VS Code commands with Ctrl+Shift+P / Cmd+Shift+P



Activity Bar

Quick access to different views and extensions



Panels

Terminal, Problems, Output, and Debug Console

Demo-Navigator + Breadcrumbs help you navigate complex projects efficiently.

Keyboard Shortcut Mastery

Command	Windows/Linux	Mac
Command Palette	Ctrl+Shift+P	⌘⇧P
Quick Open	Ctrl+P	⌘P
Toggle Terminal	Ctrl+`	^`
Multi-Cursor	Alt+Click	⌥Click
Format Document	Shift+Alt+F	⇧⌥F

PDF-Poster zum Ausdrucken with Top 10 shortcuts for all platforms.

Multi-Cursor & Snippets

Multiple Selection Methods

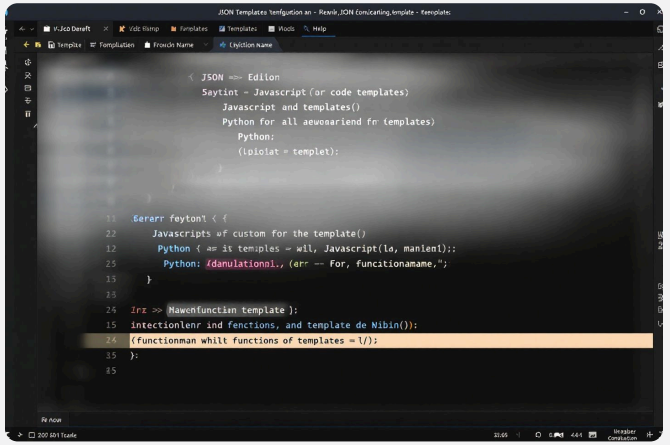
Alt-Click / ⌥Click, Column Select

Different ways to edit multiple lines at once

Custom Snippets

Eigene Snippets in javascript.json

Create reusable code templates



Define your own snippets to boost productivity

Settings Sync & Themes

GitHub-Login zum Synchronisieren

Sync settings across multiple devices

Popular Themes

Beliebte Themes 2025: One Dark Pro, GitHub Light

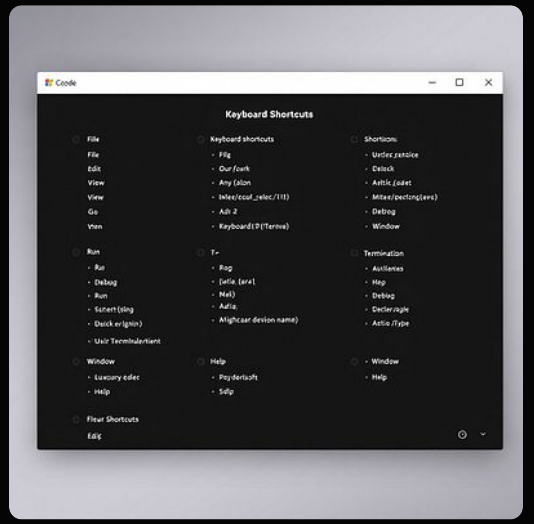
Customization

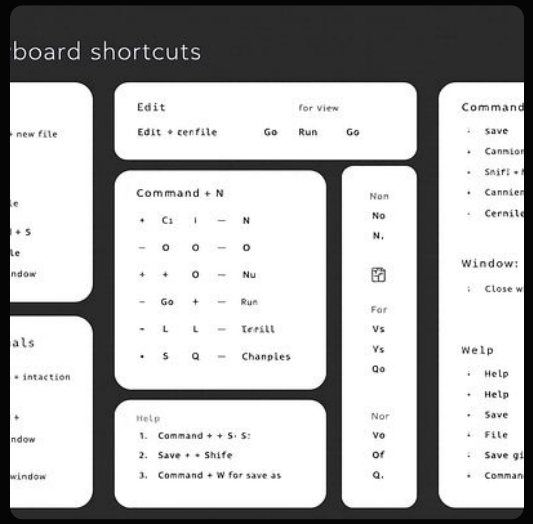
Personalize your editor appearance

What Gets Synced

Settings, keybindings, snippets, and extensions

Module 5 Quiz & Cheatsheets

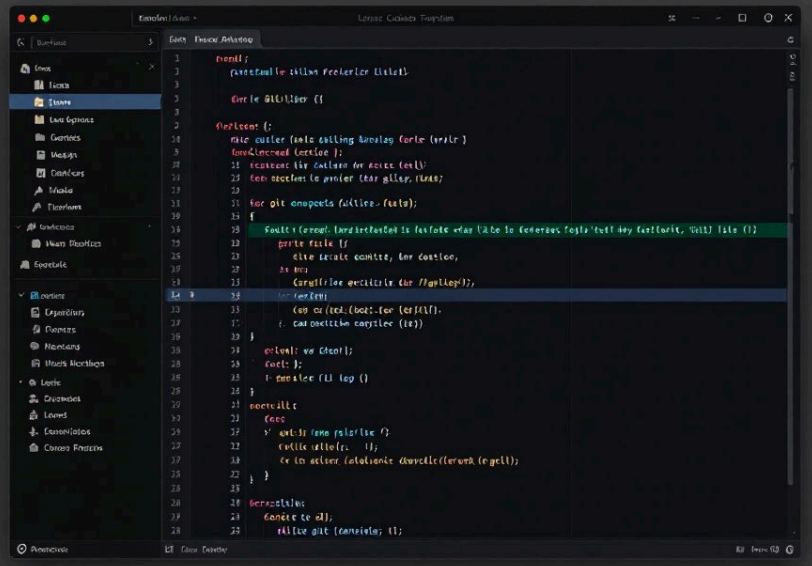


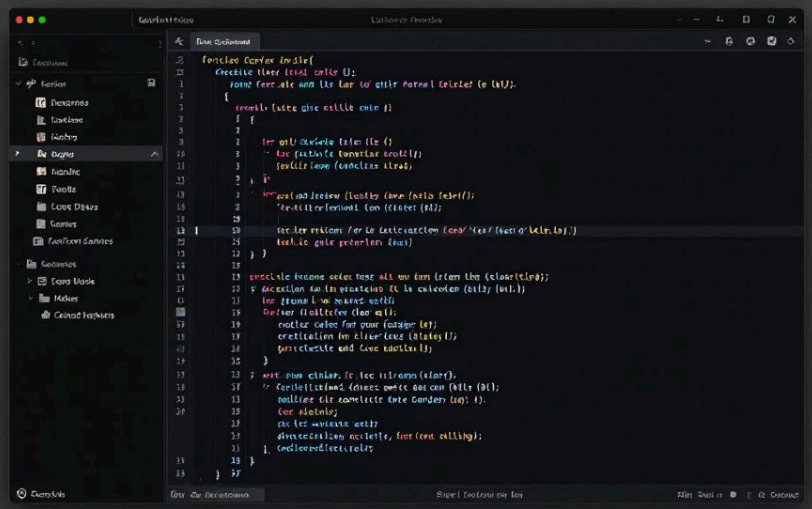






Tastatur-Challenge (Selbst-Test). JSON-Snippet-Galerie Download.





Terminal Deep-Dive (Module 6)

Open & Configure

Öffnen, Split, Search terminal functionality

Git Integration

Demo: Git-Workflow entirely in Editor

Terminal Shortcuts

Keyboard commands for efficient terminal use

Customization

Terminal appearance and behavior settings

Shell Profiles & Environment Files

Terminal Profiles

"terminal.integrated.profiles" Erklärung

Configure different shell environments

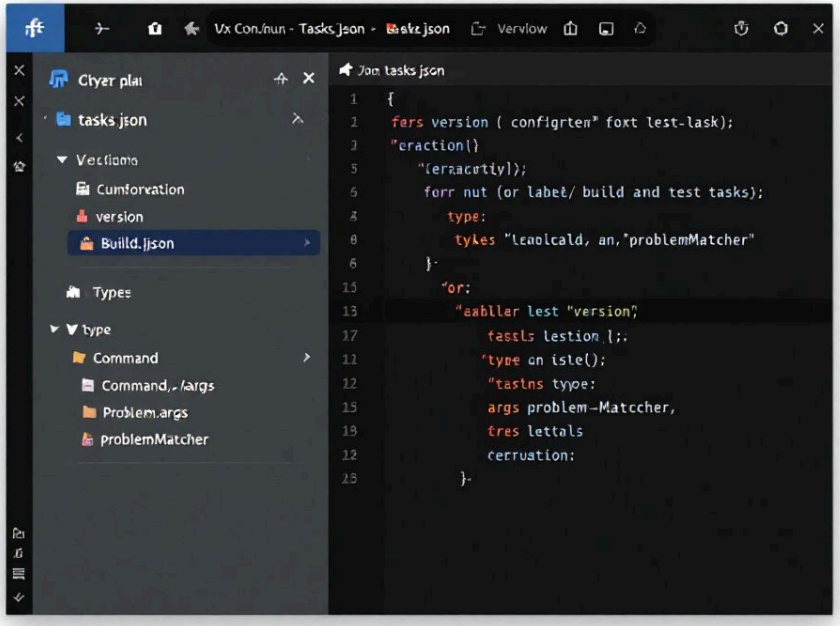
Environment Variables

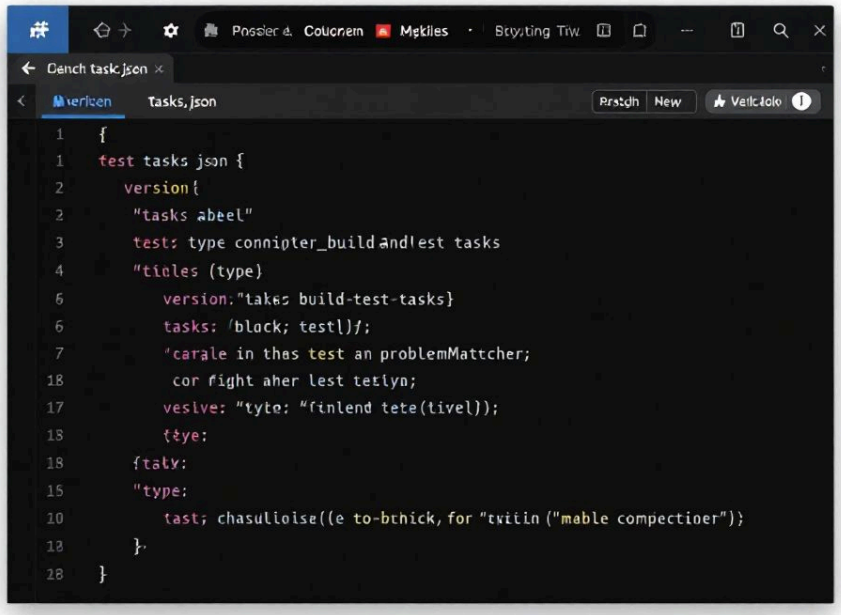
.env + Secret-Storage

Manage sensitive configuration securely

```
{

"terminal.integrated.profiles.windows": {
    "PowerShell": {
      "source": "PowerShell",
      "icon": "terminal-powershell"
    },
    "Git-Bash": {
      "path": "C:\\Program Files\\Git\\bin\\bash.exe"
    }
  }
}
```





Automated Tasks

Task Configuration

tasks.json Struktur + Beispiel NPM test

Define automated workflows for your project

Build Integration

Connect tasks to build systems

Automate compilation and packaging

Problem Detection

Video zeigt Build & Problems-Panel

Automatically identify and navigate to issues

Quick-Linting-Pipeline



Configure Linter

ESLint setup in project

Create Task

Define linting task in tasks.json

Run Analysis

Execute linting task

Review Issues

Problems Pane shows results

Screencast 4 Min demonstrates the complete linting workflow.

Module 6 Recap & Mini-Quiz

- Question 1: How do you define a custom terminal profile in VS Code?

  > By configuring the "terminal.integrated.profiles" setting in settings.json with the shell path and options.

- Question 2: What file is used to define automated tasks in VS Code?

  > tasks.json in the .vscode folder of your project.

- Question 3: How can you view problems detected by linters in VS Code?

  > In the Problems panel, which can be opened from the View menu or with Ctrl+Shift+M / Cmd+Shift+M.

- Question 4: How do you create a split terminal in VS Code?

  > By clicking the split button in the terminal panel or using the keyboard shortcut Ctrl+Shift+5 / Cmd+\.

4 Task-Fragen, Lösungen im Repository.

Debugging Fundamentals (Module 7)

Core Debugging Tools

Breakpoints, Watch, Call-Stack

Essential components for effective debugging

Interface Navigation

UI-Rundgang im Video

Learn to navigate the debugging interface

Debug Actions

Step Over, Step Into, Step Out

Control execution flow during debugging

Python Debugging Flow

Configure Debugger

Launch Config, justMyCode=false

Set Breakpoints

Mark points where execution should pause

Start Debugging

Launch the debug session

Inspect Variables

Variables Explorer Demo

Node Debugging & Hot-Reload

Continuous Development

nodemon + Auto-Attach

Keep your application running while making changes

React Integration

React HMR Breakpoint Video

Debug React applications with Hot Module Replacement

Configuration

Setup for automatic debugging attachment

Optimize your Node.js debugging workflow



Log-Points & Inline Values

Non-Stop Debugging

Nicht-stoppen-debuggen

Inspect values without pausing execution

Dynamic Inspection

Beispiel: Inspect variable on the fly

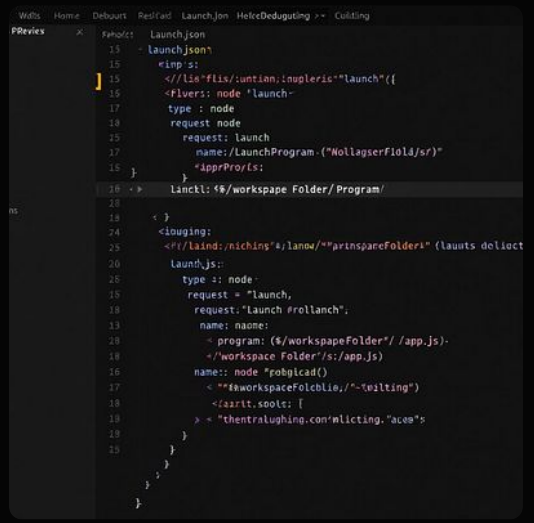
See values as your code runs

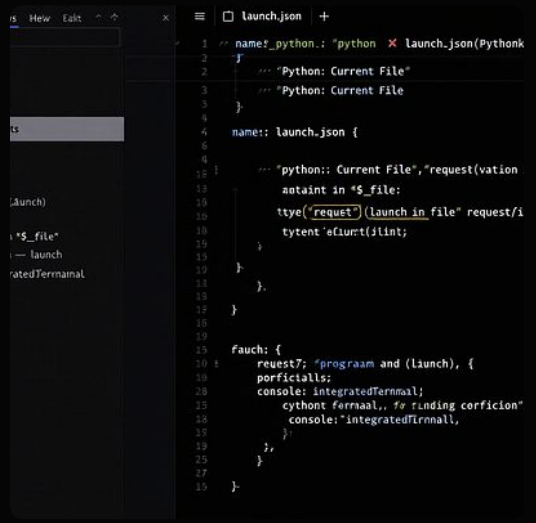
Implementation

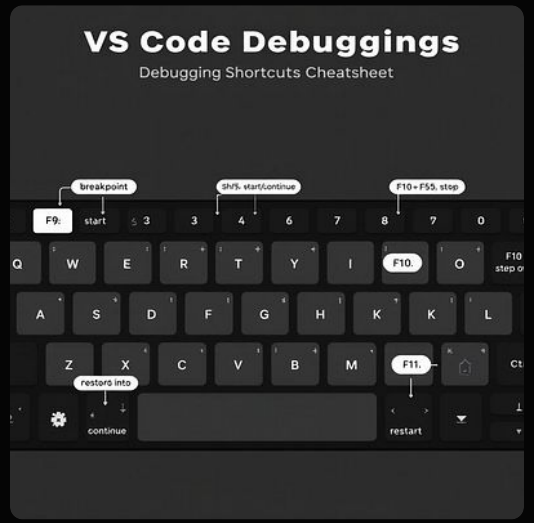
How to set up and use logpoints

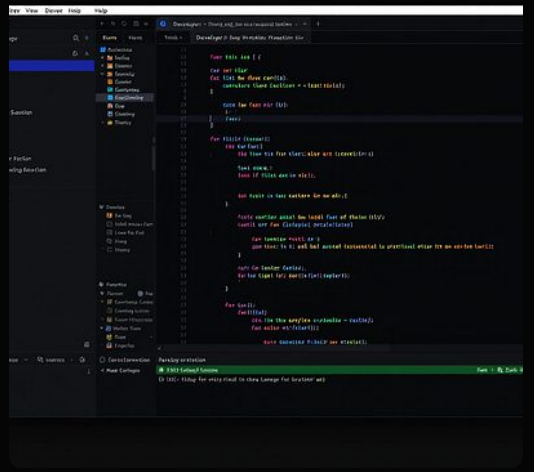
Configure inline value display

Module 7 Quiz & Resources









Sample launch.json Downloads. Debug-Shortcut-PDF.

Web-Preview & Live Server (Module 8)



Real-time Updates

Live-Reload for HTML/CSS/JS



One-Click Launch

"Go Live" Button Demo



Browser Integration

Preview your web pages instantly



Configuration

Customize server settings

Browser-Preview in VS Code

Integrated Preview

WebView Panel öffnen

View web content without leaving VS Code

Port Forwarding

Ports-Mapping Demo

Connect to local development servers

Multi-Device Testing

Test responsive designs

Simulate different screen sizes

Static-Site Deployment to GitHub Pages

Configure Action

Action "deploy-pages"

Setup Workflow

Workflow YAML kurz erläutern

Commit Changes

Push configuration to repository

View Deployment

Access published website

CSS-Sprint: Styling in Echtzeit

Edit CSS

Make styling changes

Save File

Trigger auto-reload



Refine Design

Iterate quickly

View Changes

See updates instantly

Video: Tailwind-Änderung ↔️ Live-Reload. Teilnehmer pausieren & nachbauen.

Module 8 Recap & Cheats

- Question 1: How do you start Live Server in VS Code?

  > Click the "Go Live" button in the status bar or right-click an HTML file and select "Open with Live Server".

- Question 2: What GitHub Action is used for deploying static sites to GitHub Pages?

  > The "deploy-pages" action.

- Question 3: How can you view a website inside VS Code?

  > Using the Browser Preview extension or the built-in Simple Browser.

- Question 4: What is the benefit of using Tailwind with Live Reload?

  > It allows you to see styling changes instantly without manually refreshing the browser.

- Question 5: How do you configure port forwarding in VS Code?

  > Through the Ports panel in the Remote Explorer or using the "Forward a Port" command.

PDF "Frontend Quickstart". Selbst-Quiz (5 Fragen).

AI Add-ons Comparison (Module 9)

Feature	GitHub Copilot	Amazon Q	Tabnine	Cody
Code Completion	✓	✓	✓	✓
Chat Interface	✓	✓	Limited	✓
Security Scanning	Limited	✓		Limited
On-device Option			✓	✓
Language Support	Extensive	Good	Good	Good

Tabelle: Copilot | Amazon Q | Tabnine | Cody. Sprachen-Support & Pricing.

Amazon Q Demo

Code Assistance

IntelliSense integration

Context-aware code suggestions

Security Features

Security-Scan capabilities

Identify potential vulnerabilities

AWS Integration

Specialized for AWS services

Cloud-specific recommendations

Video 3 Min demonstrates key features and workflows.

Tabnine & Cody Hands-on

Local Processing

On-device Model vs Cloud

Privacy and performance considerations

Repository Awareness

Repo-Aware Chat Demo

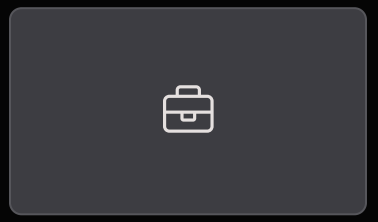
Context from your codebase

Unique Features

Specialized capabilities of each tool

When to choose one over another

Capstone Project & Next Steps (Module 10)



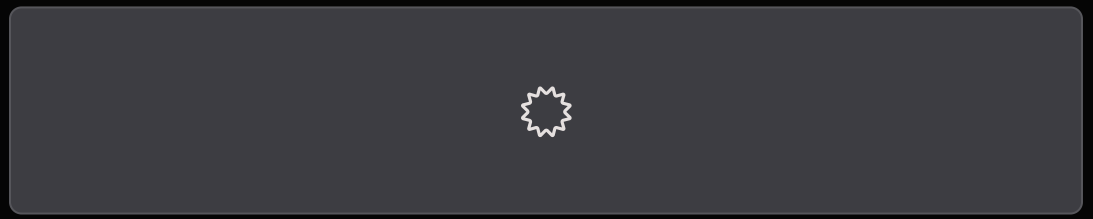
Project Briefing

Projekt-Briefing: Baue Mini-App, push in eigenes Repo

2

Submission

Abgabe via Pull Request; Muster-Tests laufen in Action



Certification

Zertifikat nach bestandener Checkliste

Danke fürs Mitmachen & Weiterlernen!